Issues in the Design of a Code Generator and Target Machine

Introduction

In compiler construction, **code generation** is the final phase of compilation.
It converts the intermediate code (produced by the previous stages) into **machine code** or **assembly code** that can be executed by the **target machine**.

The design of a good code generator is not easy because it must produce **efficient, correct, and optimized machine code**.
Several issues arise while designing the code generator and choosing the target machine.

---

1. Input to the Code Generator

The **intermediate code** is the input to the code generator.
It may be in the form of **three-address code**, **syntax tree**, or **DAG (Directed Acyclic Graph)**.
The generator must handle all possible forms and translate them accurately into target code.

---

2. Target Machine Model

Before generating code, the compiler designer must understand the **target machine architecture**, such as:

- Number of **registers** available
- **Addressing modes** supported
- **Instruction set** (RISC or CISC)
- **Word length** (8-bit, 16-bit, 32-bit, etc.)

A mismatch between intermediate code and machine architecture can lead to **inefficient code**.

---

3. Memory Management

The code generator must decide:

- How to **allocate memory** for variables, arrays, and functions.
- How to **map variables** to registers or memory locations.
- How to **access data efficiently** during execution.

Efficient memory usage improves both speed and performance.

---

4. Register Allocation

Registers are faster than memory, but they are **limited in number**.
So the code generator must:

- Decide **which variables** to keep in registers.
- **Reuse registers** wherever possible.
- Minimize **memory load/store operations**.

Good register allocation is key to generating **optimized code**.

---

5. Instruction Selection

Different machine architectures have different instruction sets.
The code generator must:

- Choose the **best machine instruction** for each intermediate operation.
- Use **complex instructions** if they perform multiple operations efficiently.
- Ensure **correctness and compatibility** with the target machine.

---

6. Instruction Ordering (Sequencing)

The order of instructions affects performance.
The generator must:

- Arrange instructions to **reduce pipeline stalls**.
- Avoid **unnecessary jumps** and **branch delays**.
- Ensure **dependencies** are handled properly.

A well-ordered sequence results in **faster execution**.

---

7. Code Optimization

Generated code should be **optimized** for:

- **Speed** (fast execution)
- **Memory** (less storage)

Techniques like **common subexpression elimination**, **loop optimization**, and **dead code removal** may be applied during or after code generation.

---

8. Error Handling

If errors exist in earlier phases, the code generator should:

- **Detect and handle** them gracefully.
- Generate **useful error messages**.
- Avoid producing incorrect or unsafe machine code.

Conclusion

Designing a code generator involves balancing **correctness**, **efficiency**, and **simplicity**.
It must produce optimized, executable code while respecting the **constraints of the target machine**.
A well-designed code generator ensures that the final program runs efficiently and correctly on the desired platform.

Runtime Storage Management

Introduction

When a program is executed, it needs **memory** to store:

- Instructions
- Variables
- Function calls
- Temporary values

The process of **allocating and managing this memory during program execution** is called **Runtime Storage Management**.

The **runtime storage manager** is a part of the compiler that takes care of memory allocation and deallocation when the program runs.

Main Goals of Runtime Storage Management

1. **Efficient use of memory**
   → Use limited memory space properly and avoid wastage.
2. **Automatic allocation and deallocation**
   → Allocate memory when variables are created and free it when no longer needed.
3. **Support for recursion and dynamic data structures**
   → Manage memory for function calls, arrays, linked lists, etc.

Memory Areas in a Running Program

When a program runs, its memory is divided into different areas:

1. Code Area

- Also called **text segment**.
- Stores **compiled machine code or instructions**.
- Usually **read-only** (cannot be changed at runtime).

---

2. Static Area

- Stores **global variables** and **static variables** whose size is known at compile time.
- Memory for these variables is **allocated once** and remains till the program ends.

🟢 Example:

```
int x = 10;   // Stored in static area
```

---

3. Stack Area

- Used for **function calls and local variables**.
- Each function call creates an **activation record (stack frame)** containing:
  - Function parameters
  - Local variables

    - Return address
- Memory is **freed automatically** when the function returns.

🟢 Example:
Recursive functions use the stack for each call.

---

4. Heap Area

- Used for **dynamic memory allocation** (like `malloc()` in C, `new` in C++/Java).
- The size and lifetime of variables are **decided at runtime**.
- Managed by the **programmer or garbage collector**.

🟢 Example:

```
int *p = (int*)malloc(sizeof(int));  // Stored in heap
```

---

5. Free Space

- The unused portion of memory that can be used later by **heap or stack** as needed.

Types of Runtime Storage Allocation Strategies

1. Static Allocation

- Memory is assigned **at compile time**.
- Works when the size and lifetime of variables are **known in advance**.
- Fast but not flexible.

🟢 Example: Global and static variables.

---

2. Stack Allocation

- Memory is managed using a **stack data structure**.
- Every function call **pushes** a new frame; when it returns, the frame is **popped**.
- Suitable for **recursive functions**.
- Faster and automatic, but cannot handle **dynamic data** that changes size at runtime.

---

3. Heap Allocation

- Memory is allocated **dynamically at runtime**.
- Used for data whose size or lifetime is **unknown at compile time**.
- Managed manually (`malloc`, `free`) or automatically (garbage collection).

Activation Record (Important Concept)

Each function call gets an **activation record** that stores:

- Parameters
- Return address
- Local variables
- Temporary values

The runtime system **creates and destroys activation records** on the **stack** as functions are called and returned.

Conclusion

Runtime storage management ensures that memory is:

- **Allocated efficiently**
- **Used safely**
- **Released properly**

It handles memory for variables, functions, and dynamic structures during program execution, supporting both **recursion** and **dynamic allocation**.

Basic Blocks

Introduction

A **Basic Block** is a **sequence of consecutive statements** in a program that:

- Has **only one entry point** (control enters only at the first statement)
- Has **only one exit point** (control leaves only at the last statement)

In other words, there are **no jumps or branches** except possibly at the end of the block.

Basic blocks are the **building blocks** for many compiler optimizations and code generation techniques.

Definition

A **Basic Block** is a straight-line sequence of code with no branches in except to the entry and no branches out except at the exit.

Example

Consider the following code:

```
1. a = b + c;
2. d = a * e;
3. if (d > 10)
4.     x = d - 1;
5. y = x + 5;
```

We can divide it into **basic blocks** as follows:

Basic Block 1:

```
1. a = b + c;
2. d = a * e;
```

(Sequential instructions — no jumps inside)

Basic Block 2:

```
3. if (d > 10)
```

(Conditional jump — ends the block)

Basic Block 3:

```
4. x = d - 1;
```

(Executed if condition is true)

Basic Block 4:

```
5. y = x + 5;
```

(Executed next — separate entry point)

Steps to Identify Basic Blocks

To divide a program into basic blocks:

Step 1: Identify Leaders

A **leader** is the **first statement** of a basic block.
Rules to find leaders:

1. The **first statement** of the program is a leader.
2. The **target of a jump** or branch statement is a leader.
3. Any statement **immediately following a jump or branch** is a leader.

Step 2: Form Basic Blocks

Each **leader** and all statements **until the next leader** form one **basic block**.

Flow Graph (Control Flow Graph)

Once basic blocks are identified, we can represent the program as a **flow graph**.

- Each **node** → represents a basic block
- Each **edge** → represents possible control flow (jump or branch) between blocks

Example Flow Graph:

```
[B1] → [B2]
  ↓      ↓
 [B3] → [B4]
```

This helps in **optimization and control-flow analysis**.

Uses of Basic Blocks

1. **Optimization:**
    - Simplifies **code optimization** by working within a block.
    - Techniques like **common subexpression elimination**, **dead code removal**, etc., work on basic blocks.

2. **Code Generation:**
    - Easier to generate **machine code** for straight-line blocks.
3. **Control Flow Analysis:**
    - Helps in constructing **flow graphs** for better understanding of program execution.
4. **Debugging and Testing:**
    - Easier to isolate and analyze sections of code.

Advantages

- Makes optimization simpler and structured
- Reduces complexity in code generation
- Allows accurate flow control representation

Basic Block with Flow Graph (Diagram)

Example Code:

```
1. a = b + c;
2. d = a * e;
3. if (d > 10)
4.     x = d - 1;
5. y = x + 5;
```

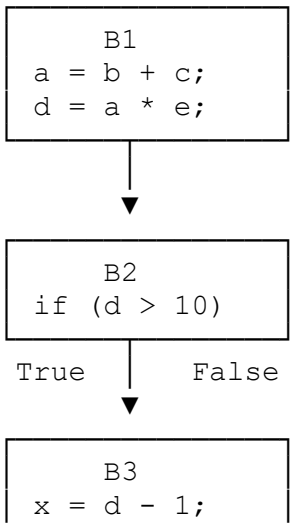
Step 1: Identify Leaders

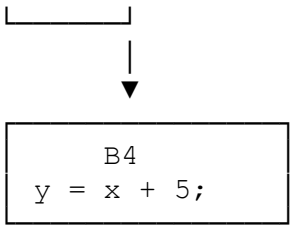
1 Statement 1 → First statement → **Leader**
2 Statement 3 → Contains a conditional jump → **Leader**
3 Statement 4 → Target of jump → **Leader**
4 Statement 5 → Follows a conditional jump → **Leader**

Step 2: Form Basic Blocks

Block Name	Statements
B1	1. a = b + c; 2. d = a * e;
B2	3. if (d > 10)
B3	4. x = d - 1;
B4	5. y = x + 5;

Step 3: Flow Graph Diagram





Explanation of Flow Graph:

- **B1 → B2:** Normal flow from first two statements to the condition.
- **B2 → B3:** If condition is true (`d > 10`), control goes to **B3**.
- **B2 → B4:** If condition is false, control skips to **B4**.
- **B3 → B4:** After excuting **B3**, control flows to **B4**.

Conclusion

Basic blocks are fundamental units of code used in **optimization** and **code generation** phases of the compiler.
They help the compiler understand program control flow and make it easier to generate efficient machine code.

Flow Graphs

Introduction

A **Flow Graph** (also called a **Control Flow Graph** or **CFG**) is a **graphical representation** of the control flow of a program.
It shows **how control moves** from one **basic block** to another during program execution.

Each **node** in the flow graph represents a **basic block**, and each **directed edge (arrow)** represents the **possible flow of control** between two blocks.

Definition

A **Flow Graph** is a directed graph where:

- Each node represents a **basic block**, and
- Each edge shows **possible control transfer** between two blocks.

Example Program

Consider the following simple program:

```
1. a = b + c;
2. d = a - e;
3. if (d > 0)
4.     x = d * 2;
5. else
6.     x = d + 2;
7. y = x - 1;
```

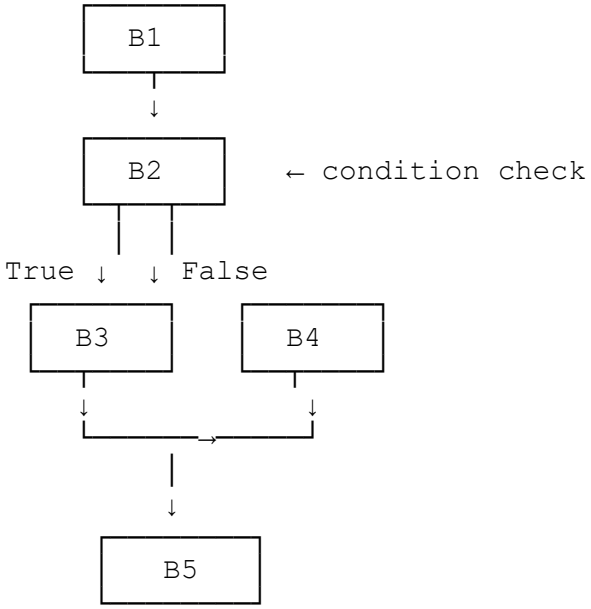
Step 1: Identify Basic Blocks

Block	Statements
B1	1. a = b + c; 2. d = a - e;
B2	3. if (d > 0)
B3	4. x = d * 2;
B4	6. x = d + 2;
B5	7. y = x - 1;

Step 2: Identify Control Flow

- **B1 → B2** (normal sequential flow)
- **B2 → B3** (if condition true)
- **B2 → B4** (if condition false / else part)
- **B3 → B5** (after true branch)
- **B4 → B5** (after false branch)

Step 3: Draw Flow Graph



Components of a Flow Graph

1. **Nodes (Vertices):**
   - Represent **basic blocks** of the program.
   - Each node contains a set of instructions.
2. **Edges (Arrows):**
   - Represent **possible control flow** between basic blocks.
   - A directed edge `B1 → B2` means control can pass from block B1 to B2.
3. **Entry Node:**
   - The **starting point** of the program (usually the first block).
4. **Exit Node:**
   - The **end of the program** or function (control leaves here).

Uses of Flow Graphs

1. **Optimization:**
   - Used in compiler optimizations like **dead code elimination**, **loop optimization**, and **constant propagation**.
2. **Control Flow Analysis:**

    - Helps to understand **program structure** and **paths of execution**.
3. **Error Detection:**
    - Useful in identifying **unreachable code** or **infinite loops**.
4. **Code Generation:**
    - Helps in efficient **machine code generation** by understanding control transfers.

Advantages

- Provides **clear visualization** of program execution.
- Helps in **detecting redundant** or **inefficient code**.
- Serves as the **foundation** for many **compiler optimizations**.

Conclusion

A **Flow Graph** is an essential tool in the **code generation** and **optimization** phases of a compiler.
It clearly shows how control moves between different parts of a program, allowing the compiler to generate efficient and optimized code.

Use of Information in a Simple Code Generator

Introduction

A **code generator** is the part of the compiler that converts **intermediate code** into **machine code** or **assembly code** that can be executed by the **target machine**.

To generate correct and efficient code, the code generator uses various **information** collected from previous phases of the compiler — such as lexical, syntax, semantic, and intermediate representations.

1. Purpose of Using Information

The **main goal** is to produce:

- **Correct machine instructions**
- **Optimized use of registers and memory**
- **Efficient execution speed**

To achieve this, the code generator uses information from several compiler tables and data structures.

2. Sources of Information Used

(a) Symbol Table Information

- Stores details about **variables, constants, and functions**.
- Contains attributes such as:
    - Data type (int, float, char, etc.)
    - Memory location or offset
    - Scope (local/global)
    - Size of data
- **Use:** Helps in generating proper **load**, **store**, and **addressing instructions**.

🟢 Example:

```
a = b + c;
```

→ Symbol table tells data types and memory addresses of `a`, `b`, and `c`.

(b) Intermediate Code Information

- The **intermediate code (3-address code or DAG)** gives a simplified representation of the program.
- Helps the code generator understand:
  - Order of operations
  - Dependencies among variables
  - Temporary variable usage

🟢 Example:

```
t1 = b + c
a = t1
```

→ The generator converts this into target instructions such as:

```
MOV R1, b
ADD R1, c
MOV a, R1
```

(c) Type Information

- From semantic analysis, the generator knows the **data types** of operands.
- Helps select **correct machine instructions**:
  - Integer addition → `ADD`
  - Floating-point addition → `FADD`

🟢 Example:

```
x = y + z;
```

→ If `x`, `y`, `z` are floats → use floating-point addition instruction.

(d) Register Information

- The code generator must decide **which variables go into registers** and which go to memory.
- Uses **register allocation** and **register descriptors** to track which registers hold which variables.

🟢 Example:

```
MOV R1, b
ADD R1, c
MOV a, R1
```

→ Register `R1` holds the result temporarily.

(e) Target Machine Information

- Includes:
  - Number of registers available
  - Instruction set architecture (RISC or CISC)
  - Addressing modes supported
  - Word size

🟢 Example:
If target machine has only 4 registers, the generator must **reuse** registers efficiently.

(f) Runtime Environment Information

- Used to manage:
  - **Stack and heap memory**
  - **Function calls** and **return addresses**
  - **Activation records**

This helps in generating code for function calls, parameters, and return values.

3. Simple Code Generation Example

Intermediate Code:

```
t1 = a + b
t2 = t1 * c
```

Generated Machine Code:

```
MOV R1, a
ADD R1, b
MOV R2, R1
MUL R2, c
```

Here:

- Symbol table gives info about variables (`a`, `b`, `c`)
- Register info assigns `R1` and `R2`
- Target info provides instructions (`MOV`, `ADD`, `MUL`)

4. Advantages of Using Proper Information

☑ Ensures **correct and efficient code generation**
☑ Reduces **memory usage** and **execution time**
☑ Simplifies **optimization** and **error handling**
☑ Allows **machine-independent** intermediate code

Conclusion

A **simple code generator** depends on accurate information from earlier compiler phases such as the **symbol table**, **intermediate code**, **type**, and **target machine details**.
Using this information properly helps the compiler generate **optimized, correct, and executable machine code**.

Issues in Register Allocation

Introduction

In a computer system, **registers** are the **fastest memory locations** used to store temporary data and variables during program execution.

During **code generation**, the compiler must decide:

- Which values should be kept in registers
- When to move data between registers and memory

This process is called **Register Allocation**.

Because the number of registers is **limited**, several issues arise while assigning them efficiently.

1. Meaning of Register Allocation

Register Allocation is the process of assigning a large number of program variables to a small number of available CPU registers.

If not done properly, the compiler may frequently **load and store data** between registers and memory, which **slows down execution**.

2. Importance of Register Allocation

- Registers provide **faster data access** than memory.
- Efficient register use leads to **optimized machine code**.
- Reduces the **execution time** and **memory traffic**.

3. Major Issues in Register Allocation

(1) Limited Number of Registers

- Processors have a **small number of registers** (e.g., 8, 16, or 32).
- The compiler often needs to store more variables than available registers.
- It must choose **which variables stay in registers** and which go to **memory**.

🟢 *Example:*
If there are 3 registers and 5 variables, two must be stored in memory — leading to slower access.

(2) Register Spilling

- When registers are **not sufficient**, some values are **moved (spilled)** to memory.
- This increases **load and store instructions**, making the code slower.
- The compiler must carefully decide **which values to spill** to reduce performance loss.

🟢 *Example:*

```
MOV R1, A
MOV R2, B
MOV R3, C
; no free register — must spill one value
```

(3) Register Assignment Problem

- Once registers are selected, they must be **assigned efficiently** to minimize reloading.
- The compiler tries to keep **frequently used variables** in registers longer.

🟢 *Example:* Variables used repeatedly in loops should stay in registers.

(4) Interference Between Variables

- Two variables that are **alive at the same time** cannot share the same register.
- The compiler uses **liveness analysis** and **interference graphs** to detect such conflicts.

🟢 *Example:*
If both `x` and `y` are needed at the same time, they must be in different registers.

(5) Function Calls and Register Saving

- During a function call, registers may be used by both caller and callee.
- The compiler must decide:
  - Which registers should be **saved and restored**
  - Which are **temporary (volatile)**
- This ensures values are not accidentally overwritten.

(6) Different Register Classes

- Some architectures have **separate registers** for:
  - Integer operations
  - Floating-point operations
  - Address calculations
- The compiler must allocate variables to the **right register class**.

(7) Register Allocation Across Basic Blocks

- Register allocation is easier within a **single basic block**,
  but across multiple blocks or functions, the **lifetimes** of variables overlap.
- The compiler must handle this global allocation carefully for efficiency.

4. Techniques Used to Handle These Issues

- **Graph Coloring Algorithm** → For global register allocation
- **Priority-based Allocation** → Keeps most-used variables in registers
- **Spill Cost Estimation** → Chooses least costly variable to spill
- **Live Range Splitting** → Shortens variable lifetimes to reduce interference

5. Conclusion

Register allocation is one of the most **important and challenging** tasks in code generation.
The main issues include **limited registers**, **spilling**, **variable interference**, and **function call handling**.
A good allocation strategy improves **execution speed** and **reduces memory usage**.

Assignment and Basic Blocks

Introduction

In the **code generation** phase of a compiler, the source program is broken into smaller manageable parts called **basic blocks**, and then **assignment statements** inside these blocks are translated into **machine instructions**.

Both **assignments** and **basic blocks** are essential for generating efficient, optimized code.

1. Assignment Statements

An **assignment statement** is one of the most common forms of instructions in a program.
It **assigns a value or result of an expression** to a variable.

General Form:

```
x = y op z
```

where

- `x` → result variable (left-hand side)
- `y`, `z` → operands (right-hand side)
- `op` → operator (+, -, *, /, etc.)

Example:

```
a = b + c
d = a * e
```

These are **assignment statements** that perform arithmetic and store results.

2. Translation of Assignment Statements

During **code generation**, each assignment is translated into **machine or assembly instructions**.

Example Translation:

Intermediate code:

```
t1 = b + c
a = t1
```

Generated machine code:

```
MOV R1, b
ADD R1, c
MOV a, R1
```

👉 Here,

- `MOV` → moves data into registers
- `ADD` → performs addition
- `a`, `b`, `c` → variables obtained from the **symbol table**

So, each **assignment** is converted into a sequence of **low-level instructions**.

3. Basic Blocks

Definition

A **basic block** is a **sequence of consecutive statements** in which:

- Control enters only at the **beginning**, and
- Control leaves only at the **end** (no jumps inside the block).

It is a **straight-line sequence of code** with **single entry and exit points**.

Steps to Identify Basic Blocks

1. **Find Leaders:**
    - The **first statement** is a leader.
    - The **target** of a jump is a leader.
    - The **statement after a jump** is a leader.
2. **Form Basic Blocks:**
    - Each leader and the statements that follow it (until the next leader) form a **basic block**.

Example:

```
1. a = b + c;
2. d = a * e;
3. if (d > 10)
4.     x = d - 1;
5. y = x + 5;
```

☑ **Basic Blocks:**

- **B1:** (1–2)
- `a = b + c;`
- `d = a * e;`
- **B2:** (3)
- `if (d > 10)`
- **B3:** (4)
- `x = d - 1;`
- **B4:** (5)
- `y = x + 5;`

4. Relationship Between Assignment and Basic Blocks

- **Assignment statements** are the main content inside **basic blocks**.
- Each **basic block** is made up of one or more **assignment statements** and possibly one **control statement** (like if, goto).
- Basic blocks make it easier for the compiler to:
  - Perform **optimizations** (like removing redundant assignments).
  - Generate **efficient machine code** for straight-line code.
  - Analyze **data flow** and **control flow**.

5. Example:

Intermediate code:

```
1. t1 = a + b
2. t2 = t1 * c
3. if t2 < 10 goto L1
4. t3 = t2 - 5
5. L1: d = t3 + 2
```

☑ **Basic Blocks:**

- **B1:** (1–2)
- **B2:** (3)
- **B3:** (4–5)

Each block contains **assignment statements** that the code generator converts to **machine code**.

6. Importance

- **Assignments** → define actual computations
- **Basic Blocks** → define logical grouping for optimization and code generation

Together, they help in:

- **Simplifying code analysis**
- **Improving performance**
- **Generating efficient executable code**

Conclusion

Assignment statements describe the **computations** in a program, and **basic blocks** organize these computations into simple, straight-line sequences for easier **optimization** and **code generation**.
Thus, both are fundamental concepts in the **code generation** phase of compiler construction.

Code Generation from DAG and Dynamic Code Generation Algorithm

1. Introduction

During the **code generation** phase of a compiler, the goal is to convert **intermediate code** (like 3-address code) into **machine code** or **assembly code**.

To make this process **efficient**, the compiler often uses a **DAG (Directed Acyclic Graph)** representation of expressions.

The DAG helps the compiler to:

- Detect **common subexpressions**
- Avoid **redundant computations**
- Generate **optimized code**

2. What is a DAG?

DAG (Directed Acyclic Graph) is a graphical representation of expressions where:

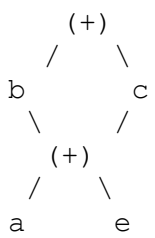
- **Nodes** represent **operations** or **operands**
- **Edges** represent **dependencies** between operations
- The graph has **no cycles** (it's acyclic)

Example:

Consider the expression:

```
a = b + c
d = a + e
```

DAG Representation:



Here:

- The subexpression `(b + c)` is used in both assignments.
- The DAG helps us **reuse** the computed value instead of recomputing it.

3. Code Generation from DAG

The process of generating machine code from DAG involves **traversing the DAG** and producing **instructions** in an order that respects dependencies.

Steps for Code Generation from DAG:

Step 1: Identify Leaf Nodes

- Leaf nodes represent **operands** (variables or constants).
- Load them into registers or memory.

Step 2: Process Internal Nodes

- Internal nodes represent **operations** (like +, -, *, /).
- Generate code for their **children first**, then for the operation itself.

Step 3: Reuse Computed Values

- If a subexpression is used multiple times, compute it **once** and reuse the result.

Step 4: Generate Machine Instructions

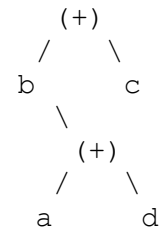
- Convert each node into corresponding **machine-level instructions**.

Example:

Expression:

```
t1 = b + c
t2 = a + t1
t3 = t1 + d
```

DAG:



Generated Code:

```
MOV R1, b
ADD R1, c        ; R1 = b + c (t1)
MOV R2, a
ADD R2, R1       ; R2 = a + t1 (t2)
MOV R3, R1
ADD R3, d        ; R3 = t1 + d (t3)
```

☑ The subexpression `(b + c)` is calculated **once** and reused.

4. Dynamic Code Generation Algorithm

The **Dynamic Code Generation Algorithm** is used when the compiler **generates code during runtime** or dynamically selects the best sequence of instructions **based on DAG traversal**.

It works by **recursively processing** the nodes of the DAG and **assigning registers** dynamically.

Algorithm:

```
Procedure Generate(node):
1. If node is a leaf (operand):
      return location of operand
2. Else:
      left = Generate(left child)
      right = Generate(right child)
3. Choose a register R to store the result
4. Generate code: OP R, left, right
5. Return R
```

Explanation:

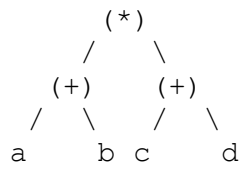
- The algorithm works **recursively** from leaves to root.
- Each operator node generates instructions **after its children are processed**.
- Registers are **dynamically assigned** as needed.
- When a subexpression is already computed, it reuses the register (no recomputation).

Example:

Expression:

```
x = (a + b) * (c + d)
```

DAG:



Dynamic Code Generation Steps:

1. Generate code for `(a + b)`
   → `MOV R1, a`
   → `ADD R1, b`
2. Generate code for `(c + d)`
   → `MOV R2, c`
   → `ADD R2, d`
3. Multiply both results
   → `MUL R1, R2`

☑ Final Generated Code:

```
MOV R1, a
ADD R1, b
MOV R2, c
ADD R2, d
MUL R1, R2
MOV x, R1
```

5. Advantages of DAG-Based Code Generation

- Removes **redundant calculations**
- Reduces **number of instructions**
- Efficient **register usage**
- Generates **optimized machine code**

6. Conclusion

DAG-based code generation is an effective method to produce **optimized and efficient machine code**.
The **Dynamic Code Generation Algorithm** uses recursive traversal of DAG nodes to produce instructions dynamically, ensuring **correct order of operations** and **efficient register management**.